
领悦数字

API Functional 测试指南

修订记录

[illegible]

目 录

- 1. 导言..... 4
 - 1.1 目的..... 4
 - 1.2 范围..... 4
 - 1.3 术语定义..... 4
- 2. 测试需求分析..... 4
- 3. 测试计划制定..... 5
- 4. 测试用例设计与脚本开发..... 5
 - 4.1 梳理接口清单..... 5
 - 4.2 用例设计与脚本开发..... 5
- 5. 测试执行..... 6
 - 5.1 冒烟测试..... 6
 - 5.2 功能测试..... 6
 - 5.3 回归测试..... 7
- 6. 测试总结..... 7

1. 导言

1.1 目的

提供 API Functional 测试的方法,为 API Functional 测试学习与实施提供指导,提升 API Functional 测试的水平与测试效率;通过 API Functional 测试,及早发现接口问题,有效提升软件开发效率及产品质量,有效节省人力成本和时间成本投入。

1.2 范围

领悦数字 API Functional 测试

1.3 术语定义

API Functional 测试,指以 API 为测试对象,检测集成后的系统组件或者系统之间的主要功能和业务流程是否正确的测试。

2. 测试需求分析

API Functional 测试需求的生成需参照接口需求文档,同时要把开发输出的接口定义文档为测试需求的主要输入。

测试人员需要明确并确定接口定义,并判断接口定义是否明确,主要分析请求和响应。

请求:

- ◆ 请求 url 是否合理
- ◆ 请求信息头是否明确
- ◆ 请求方法是否合理
- ◆ 请求参数是否必填、含义、参数方式 (url\form\json)

响应:

- ◆ 响应状态码是否合理
- ◆ 响应数据格式是否明确
- ◆ 响应信息头内容是否明确

异常情况：异常返回信息和错误代码

3. 测试计划制定

测试人员需根据项目需求制定测试计划以及测试策略。其内容可包括以下方面：

- ◆ 测试策略，包括回归策略；
- ◆ 接口测试的范围；
- ◆ 测试环境；
- ◆ 测试工作量预估；
- ◆ 测试人员的任务分配；
- ◆ 测试进度安排；
- ◆ 测试过程中可能遇到的风险分析和风险预防；

测试人员需依据《领悦数字【通用】测试计划》进行制定。

4. 测试用例设计与脚本开发

测试人员根据接口定义文档准备测试用例，并进行脚本开发。

4.1 梳理接口清单

测试人员根据接口文档或开发提供的 postman 脚本（包含本地迭代新增及变更接口）梳理接口清单。

4.2 用例设计与脚本开发

接口测试是无界面的功能测试，设计用例思路跟功能测试一样，功能测试注重的是测前端页面，接口测试注重的是测后端接口。具体测试用例设计方法可参考《领悦数字

测试用例设计指南》。

(1) 输入参数测试：针对输入的参数进行测试，也可以说是假定接口参数的不正确性进行的测试，确保接口对任意类型的输入都做了相应的处理，包括输入参数合法、输入参数不合法、输入参数为 null，输入参数超长等；

(2) 功能测试：测试这个接口的功能是否实现，并且测试这个接口是否按照接口文档来进行开发；

(3) 异常场景：测试接口对异常场景的处理情况。如：请求超时、快速连续点击、请求失败情况（任务型的，失败后是否可以重新下发任务）。

测试用例模板见如下附件：



接口用例模板.xlsx

接口测试用例评审需要产品、开发、测试人员共同参与。通过用例评审，验证大家对需求的理解是否有出入，同时对用例设计提出不同的意见。测试人员根据用例评审的意见，进行接口测试用例调整，并维护在 JIRA/CF 中。

测试人员可根据设计好的测试用例，选择合适的接口测试工具，进行脚本开发。常用的接口测试工具包括 postman、jmeter。

5. 测试执行

5.1 冒烟测试

测试人员可根据设计好的接口测试用例，选取正向用例进行冒烟测试。

若大部分接口可正常调通，则可展开正式的接口测试，依照测试计划展开测试执行；若大部分接口调不通，该被测版本将被退回开发组，由开发人员修复缺陷后，再次冒烟测试。

5.2 功能测试

测试人员依照确定的测试计划展开测试执行活动。测试执行主要在 INT 环境和 SGT

环境。根据测试用例的操作步骤进行接口测试操作，查看不同的参数请求及接口的返回的数据是否达到预期，并将接口测试执行结果回填到 JIRA/CF 上，若发现问题则提交缺陷并跟踪解决。提交缺陷流程可参考《领悦数字 JIRA 测试使用指南》。

5.3 回归测试

回归测试是指修改了缺陷后，重新进行测试以确认修改没有引入新的错误。

回归测试策略通常有以下两种：

- (1) 可以选择完全重复测试，即全量回归。缺陷修复后重新将把所有的测试用例执行一遍，以确认问题修改的正确性和修改后相关功能是否受到影响。
- (2) 可以选择性重复测试，即增量回归。缺陷修复后选择一部分测试用例执行，以确认问题修改的正确性和修改后周边是否受到影响，包含如下方法：
 - ◆ 覆盖修改法，针对发生错误的模块，选取这个模块的全部用例进行测试。这样只能验证本模块是否还存在缺陷，但不能保证周边与它有联系的模块不会因为这次改动而引发缺陷。
 - ◆ 周边影响法，除了把出错模块的用例执行之外，把周边和它有关联的模块的用例也执行一遍，保证回归测试的质量。
 - ◆ 指标达成法，在测试全部完成后，看看有没有达到既定的指标。如对修改的部分代码 100% 的覆盖，对关联接口 60% 的覆盖等。

测试过程中发现的缺陷要及时记入 Jira 中，并对修复后的缺陷进行验证，直至缺陷关闭。

6. 测试总结

通过对测试过程和结果进行总结，生成测试报告。测试报告中需体现测试的范围、时间、发现的缺陷以及缺陷状态，以及测试结论等，对于存在未关闭缺陷的情况要进行测试风险分析，进行风险预警。

测试报告可通过 Jira 自动生成，并在此基础上进行编辑与完善，体现测试需要总结的内容。

JIRA 中自动生成测试报告操作步骤可参考《领悦数字 JIRA 测试操作指南》。